

Web Programming Lab

Python Basics

Adi Ramanarayan

230905380 48

Batch A2

Homework Questions:

Q1-3. Python program to implement first 3 homework questions.

- 1. Smallest element from list in $O(n)$.**
- 2. Bubble Sort.**
- 3. Matrix multiplication with error handling.**

Program:

```
def find_smallest(arr):  
    if not arr:  
        raise ValueError("Cannot find smallest element in an empty list")  
  
    smallest = arr[0]  
    for element in arr[1:]:  
        if element < smallest:  
            smallest = element  
  
    return smallest
```

```

def bubble_sort(arr):
    n = len(arr)

    for i in range(n):
        swapped = False

        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr


def multiply_matrices(matrix1, matrix2):
    if not matrix1 or not matrix2:
        raise ValueError("Matrices cannot be empty")

    rows1 = len(matrix1)
    cols1 = len(matrix1[0])
    rows2 = len(matrix2)
    cols2 = len(matrix2[0])

    if cols1 != rows2:
        raise ValueError(f"Cannot multiply matrices: columns of first
matrix ({cols1}) "
                        f"must equal rows of second matrix ({rows2})")

```

```

result = [[0 for _ in range(cols2)] for _ in range(rows1)]

for i in range(rows1):
    for j in range(cols2):
        for k in range(cols1):
            result[i][j] += matrix1[i][k] * matrix2[k][j]

return result

def print_matrix(matrix):
    for row in matrix:
        print(" ".join(f"{elem:6}" for elem in row))

def main():
    while True:
        print("\n" + "="*50)
        print("Select an option:")
        print("1. Find smallest element in a list")
        print("2. Bubble sort a list")
        print("3. Multiply two matrices")
        print("4. Exit")
        print("="*50)

        try:
            choice = int(input("Enter your choice (1-4): "))

            if choice == 1:
                print("\n--- Find Smallest Element ---")
                n = int(input("Enter the number of elements: "))
                arr = []
                print("Enter the elements:")

```

```

        for i in range(n):
            arr.append(float(input(f"Element {i+1}: ")))

        smallest = find_smallest(arr)
        print(f"\nThe smallest element is: {smallest}")

elif choice == 2:
    print("\n--- Bubble Sort ---")
    n = int(input("Enter the number of elements: "))
    arr = []
    print("Enter the elements:")
    for i in range(n):
        arr.append(float(input(f"Element {i+1}: ")))

    print(f"\nOriginal list: {arr}")
    sorted_arr = bubble_sort(arr.copy())
    print(f"Sorted list: {sorted_arr}")

elif choice == 3:
    print("\n--- Matrix Multiplication ---")

    rows1 = int(input("Enter number of rows for matrix 1: "))
    cols1 = int(input("Enter number of columns for matrix 1:
"))

    print("Enter elements of matrix 1:")
    matrix1 = []
    for i in range(rows1):
        row = []
        for j in range(cols1):
            row.append(float(input(f"Element [{i+1}][{j+1}]:
")))

        matrix1.append(row)

```

```

        rows2 = int(input("\nEnter number of rows for matrix 2: "))
        cols2 = int(input("Enter number of columns for matrix 2:
"))

        print("Enter elements of matrix 2:")
        matrix2 = []
        for i in range(rows2):
            row = []
            for j in range(cols2):
                row.append(float(input(f"Element [{i+1}][{j+1}]:
")))

            matrix2.append(row)

        result = multiply_matrices(matrix1, matrix2)

        print("\nMatrix 1:")
        print_matrix(matrix1)
        print("\nMatrix 2:")
        print_matrix(matrix2)
        print("\nResult (Matrix 1 × Matrix 2):")
        print_matrix(result)

    elif choice == 4:
        print("\nExiting program. Goodbye!")
        break

    else:
        print("Invalid choice! Please enter a number between 1 and
4.")

except ValueError as e:
    print(f"\nError: {e}")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")

```

```
if __name__ == "__main__":  
    main()
```

Output:

1.

```
PS C:\WorkArea\Academics\Manipal\6th Semester\Semester6Labs> python3 WPL/Lab4/hwq1.py  
  
=====br/>Select an option:  
1. Find smallest element in a list  
2. Bubble sort a list  
3. Multiply two matrices  
4. Exit  
=====br/>Enter your choice (1-4): 1  
  
--- Find Smallest Element ---  
Enter the number of elements: 5  
Enter the elements:  
Element 1: 3  
Element 2: 6  
Element 3: 9  
Element 4: 7  
Element 5: 1  
  
The smallest element is: 1.0  
  
=====
```

2.

```
Select an option:
1. Find smallest element in a list
2. Bubble sort a list
3. Multiply two matrices
4. Exit
=====
Enter your choice (1-4): 2

--- Bubble Sort ---
Enter the number of elements: 5
Enter the elements:
Element 1: 3
Element 2: 9
Element 3: 4
Element 4: 12
Element 5: 1

Original list: [3.0, 9.0, 4.0, 12.0, 1.0]
Sorted list: [1.0, 3.0, 4.0, 9.0, 12.0]
=====
```

3.

```
=====
Enter your choice (1-4): 3
```

```
--- Matrix Multiplication ---
```

```
Enter number of rows for matrix 1: 3
```

```
Enter number of columns for matrix 1: 3
```

```
Enter elements of matrix 1:
```

```
Element [1][1]: 1
```

```
Element [1][2]: 2
```

```
Element [1][3]: 3
```

```
Element [2][1]: 4
```

```
Element [2][2]: 5
```

```
Element [2][3]: 6
```

```
Element [3][1]: 7
```

```
Element [3][2]: 8
```

```
Element [3][3]: 9
```

```
Enter number of rows for matrix 2: 3
```

```
Enter number of columns for matrix 2: 3
```

```
Enter elements of matrix 2:
```

```
Element [1][1]: 9
```

```
Element [1][2]: 8
```

```
Element [1][3]: 7
```

```
Element [2][1]: 6
```

```
Element [2][2]: 5
```

```
Element [2][3]: 4
```

```
Element [3][1]: 3
```

```
Element [3][2]: 2
```

```
Element [3][3]: 1
```

```
Matrix 1:
```

```
1.0    2.0    3.0
```

```
4.0    5.0    6.0
```

```
7.0    8.0    9.0
```

```
Matrix 2:
```

```
9.0    8.0    7.0
```

```
6.0    5.0    4.0
```

```
3.0    2.0    1.0
```

```
Result (Matrix 1 × Matrix 2):
```

```
30.0    24.0    18.0
```

```
84.0    69.0    54.0
```

```
138.0   114.0    90.0
```

```
=====
```

Q4-6. Python program to implement the last 3 homework questions.

- 1. Validity of parentheses enclosed strings. Position of invalid input also recorded.**
- 2. Word by word reversal of a string. (Any continuous stream of characters classified as a word including full stop).**
- 3. Circle class.**

Program:

```
import math

class ParenthesesValidator:

    def __init__(self):
        self.opening = {'(', '{', '['}
        self.closing = {')', '}', ']'}
        self.pairs = {'(': ')', '{': '}', '[': ']'}

    def is_valid(self, s):
        stack = []

        for char in s:
            if char in self.opening:
                stack.append(char)
            elif char in self.closing:
                if not stack:
                    return False

                last_opening = stack.pop()
                if self.pairs[last_opening] != char:
                    return False

        return len(stack) == 0
```

```

def validate_with_details(self, s):
    stack = []

    for i, char in enumerate(s):
        if char in self.opening:
            stack.append((char, i))
        elif char in self.closing:
            if not stack:
                return False, f"Closing bracket '{char}' at position {i} has no matching opening bracket"

            last_opening, pos = stack.pop()
            if self.pairs[last_opening] != char:
                return False, f"Mismatched brackets: '{last_opening}' at position {pos} and '{char}' at position {i}"

    if stack:
        unclosed = ', '.join([f"'{char}' at position {pos}" for char, pos in stack])
        return False, f"Unclosed brackets: {unclosed}"

    return True, "All brackets are properly matched and closed"

class StringReverser:
    def __init__(self, string=""):
        self.string = string

    def set_string(self, string):
        self.string = string

    def reverse_words(self, string=None):
        text = string if string is not None else self.string

```

```

        words = text.split()
        reversed_words = words[::-1]
        return ' '.join(reversed_words)

def reverse_words_preserve_whitespace(self, string=None):
    text = string if string is not None else self.string

    words = text.split()
    return ' '.join(words[::-1])

def get_string(self):
    return self.string

class Circle:
    def __init__(self, radius):
        if radius < 0:
            raise ValueError("Radius cannot be negative")
        self.radius = radius

    def area(self):
        return math.pi * self.radius ** 2

    def perimeter(self):
        return 2 * math.pi * self.radius

    def set_radius(self, radius):
        if radius < 0:
            raise ValueError("Radius cannot be negative")
        self.radius = radius

    def get_radius(self):

```

```

        return self.radius

def __str__(self):
    return f"Circle(radius={self.radius:.2f})"

def main():
    while True:
        print("\n" + "="*60)
        print("Select an option:")
        print("1. Validate Parentheses/Brackets")
        print("2. Reverse String Word by Word")
        print("3. Circle Area and Perimeter")
        print("4. Exit")
        print("="*60)

        try:
            choice = int(input("Enter your choice (1-4): "))

            if choice == 1:
                print("\n--- Parentheses/Brackets Validator ---")
                validator = ParenthesesValidator()

                string = input("Enter a string with brackets (e.g., '()[]{}' or '({[]})'): ")

                is_valid, message = validator.validate_with_details(string)

                print(f"\nInput: '{string}'")
                print(f"Valid: {is_valid}")
                print(f"Details: {message}")

            elif choice == 2:

```

```

        print("\n--- String Reverser (Word by Word) ---")
        reverser = StringReverser()

        string = input("Enter a string to reverse word by word:
")

        reverser.set_string(string)

        reversed_string = reverser.reverse_words()

        print(f"\nOriginal: '{string}'")
        print(f"Reversed: '{reversed_string}'")

    elif choice == 3:
        print("\n--- Circle Area and Perimeter ---")

        radius = float(input("Enter the radius of the circle:
"))

        circle = Circle(radius)

        area = circle.area()
        perimeter = circle.perimeter()

        print(f"\n{circle}")
        print(f"Radius: {circle.get_radius():.2f} units")
        print(f"Area: {area:.4f} square units")
        print(f"Perimeter (Circumference): {perimeter:.4f}
units")

    elif choice == 4:
        print("\nExiting program. Goodbye!")
        break

    else:
        print("Invalid choice! Please enter a number between 1
and 4.")

```

```

except ValueError as e:
    print(f"\nError: {e}")
    print("Please enter valid input.")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")

if __name__ == "__main__":
    main()

```

Output:

1.

```

Select an option:
1. Validate Parentheses/Brackets
2. Reverse String Word by Word
3. Circle Area and Perimeter
4. Exit
=====
Enter your choice (1-4): 1

--- Parentheses/Brackets Validator ---
Enter a string with brackets (e.g., '()[]{}' or '{{[]}}'): ({Valid String, Invalid Parant
heses])

Input: '({Valid String, Invalid Parantheses])'
Valid: False
Details: Mismatched brackets: '{' at position 1 and ']' at position 35

=====
Select an option:
1. Validate Parentheses/Brackets
2. Reverse String Word by Word
3. Circle Area and Perimeter
4. Exit
=====

```

2.

```

Select an option:
1. Validate Parentheses/Brackets
2. Reverse String Word by Word
3. Circle Area and Perimeter
4. Exit
=====
Enter your choice (1-4): 2

--- String Reverser (Word by Word) ---
Enter a string to reverse word by word: This is a python string reverser.

Original: 'This is a python string reverser.'
Reversed: 'reverser. string python a is This'

```

3.

```

Enter your choice (1-4): 3

--- Circle Area and Perimeter ---
Enter the radius of the circle: 43

Circle(radius=43.00)
Radius: 43.00 units
Radius: 43.00 units
Radius: 43.00 units
Area: 5808.8048 square units
Perimeter (Circumference): 270.1770 units

```

```

=====
Select an option:
1. Validate Parentheses/Brackets
2. Reverse String Word by Word
3. Circle Area and Perimeter
4. Exit
=====

```

```

Enter your choice (1-4): 4

```



```

Exiting program. Goodbye!

```